Sri Lanka Institute of Information Technology

Master of Science in Information Technology

Enterprise Applications Development

Distributed Computing

Assignment : Distributed Log Aggregation and Severity Analysis System

GitHub link - https://github.com/Damithjayawardhane/Distributed-Log-Aggregation-and-Severity-Analysis-System

MS25911244

Jayawardhane BMCD

1.Use Case Derivation from Index Number

The use case assigned to this submission was determined by mapping the student index number to a predefined use case matrix provided by the module. The derivation procedure is outlined below.

1.1 Index Number

Student Index Number: [e.g. IT21XXXXXX]

The last two significant digits of the index number are extracted and used as the selection key.

1.2 Derivation Mapping

Using the modulo-based selection rule defined in the assignment briefing:

selection_key = last_two_digits mod number_of_use_cases

Example: Index = IT21123456 → last two digits = 56 → 56 mod 5 = 1 → Use Case 1

Use Case 1 corresponds to: Distributed Log Aggregation and Severity Analysis System.

1.3 Confirmation

The use case created corresponds to the problem statement presented from section 1 onward, that is, analyzing big log files on a distributed computing architecture where Mappers, Validators, and Aggregators work in a cluster topology controlled by a Coordinator. The different levels of severity include **INFO, DEBUG, WARNING, ERROR,** and **CRITICAL**.

2. Architectural Approach

The system is designed in such a way that it works as a full-fledged distributed cluster where each node executes the same code base but behaves in a different manner depending on their roles, which are defined based on command line arguments used for starting each node. Such a system removes the need to maintain multiple code bases as per the type of node involved. This approach combines two different proven models – the actor model where each node works as a separate process using only message passing, and the map-reduce model. [2], [1]

2.1 Cluster Design Overview

The cluster operates as a pipeline involving four types of nodes as follows:

Coordinator Node – Serves as the master node within the cluster. Responsibilities include handling the registration process for nodes, maintaining the topology registry, reading the log file, partitioning the file into equally sized chunks, and assigning tasks to Mapper nodes.

Mapper Nodes – Obtain their assigned log chunks, process each line with regular expressions to determine the severity level, and calculate local counts as well as unique messages.

Validator Nodes – Operate separately from Mappers. Each Validator processes the same chunk received by Mappers and verifies whether the calculated counts match those submitted by the Mapper, based on quorum consensus.

Aggregator Node – Functions as the reducer in this setup. All results validated through quorums will be aggregated here via an idempotent method of chunk-ID tracking.

2.2 Technology Stack

Component	Technology	Justification
Programming Language	Python 3.11	Rapid development, rich ecosystem
Communication Protocol	HTTP/REST (FastAPI) [17]	Simple, universal, easy to debug
Sidecar Process	Python subprocess	Lightweight, same codebase
Service Discovery	Central Registry (Coordinator)	Effective for this cluster scale
Log File Source	logpai/loghub – HDFS logs	Real-world distributed system logs
Containerization	Docker + Docker Compose[14]	Easy multi-node simulation on one machine

2.3 Node Role Assignment

Each node is launched with an environment variable ROLE=coordinator, mapper, validator, or aggregator. On startup, the shared entry point reads this variable and initialises the appropriate internal logic module. Crucially, the Sidecar process always starts regardless of role, ensuring every node in the cluster without exception has its communication, logging, retry, and circuit-breaking capabilities active from the moment it joins. [13]

3. Detailed Solution Design

3.1 Cluster Initialization and Discovery

As soon as the distributed cluster initiates, each of the non-coordinator nodes registers with the Coordinator via a POST /register HTTP request made through its respective Sidecar process. The Coordinator keeps track of live nodes in memory, which includes information about their identity, roles, IP addresses, port numbers, and time of registration. Once the required number of nodes register themselves (at least one of each type – Mapper, Validator, and Aggregator), the Coordinator then sends the cluster configuration details to all the registered nodes.

3.1.1 Discovery Mechanism

This system works using a pull approach to register nodes. Instead of using a specific tool for service discovery such as Consul and Zookeeper, each node except the Coordinator has its own configuration file set up containing the Coordinator address by the COORDINATOR_URL variable. Once started, the node will send its registration request and then keep sending heartbeats to the known address. The choice of such an approach allows avoiding any extra dependencies on infrastructure.

```
POST /register HTTP/1.1

Content-Type: application/json

{

  "node_id": "mapper-1",

  "role": "mapper",

  "host": "192.168.1.10",

  "port": 8001,

  "timestamp": "2026-05-09T10:00:00Z"

}
```

Acknowledgment consists of current known topology information, making sure the new member immediately becomes aware of other nodes. [5]

3.1.2 Dynamic Joining

In addition, the architecture must handle the addition of members to the cluster at any time in its lifetime, not just upon initialization. On receiving a registration message from a new node, the Coordinator checks the payload for correctness, registers the node, and broadcasts new topology info to all existing nodes. The latter receive a new list of peers via their Sidecars and update the routing tables accordingly. Thus, should extra mapping capabilities be required mid-execution, a new Mapper instance could be created and added to the cluster without interrupting execution of tasks.

3.1.3 Failure Detection

All active nodes are required to send a small "heartbeat" message to the Coordinator every 5 seconds. The Coordinator keeps the timestamp of the last heartbeat received from each individual node. After missing three heartbeats in a row (i.e., being offline for 15 seconds), the node will go through the corresponding state transition and be marked as a failure. [4]

Heartbeat State	Missed Count	Action Taken
HEALTHY	0	No action
DEGRADED	1–2	Log warning, begin close monitoring
FAILED	3+	Mark node dead, redistribute work

If a Mapper node is labeled as failed, the work chunk previously assigned to it that it has not yet provided results on will be queued again by the Coordinator and sent to the next available and healthy Mapper node. This process ensures that there is no loss of log data due to the failure of an individual node.

3.2 Work Chunk Assignment Strategy

Round-Robin batching is utilized by the Coordinator to split up the source log file into work chunks to be assigned to the active Mapper nodes. Prior to dispatching the work, the Coordinator scans the entire log file to determine its total number of lines. It proceeds to calculate the value of N, which represents the number of equal line ranges from the total line count divided by the number of active Mappers. Each line range is wrapped in a chunk object containing a chunk_id, line range, and log data.

3.2.1 Load Balancing Guarantee

The guarantee that Round-Robin Batching provides is the approximate equality of the shares of the total load allocated among the Map tasks with a maximum difference of no more than one line, due to the remainder after division of an integer number of log lines by the number of nodes. As a result, each Mapper has O(n/k) amount of work to do, where n denotes the total number of log lines, while k stands for the number of Mappers. Since parsing the logs is a computation-intensive task performed at a fairly constant rate, equal load translates into equal execution time for each node and eliminates bottlenecks.

3.2.2 Addition/Removal of Nodes

The described scheme ensures stable operation when there are changes to the cluster topology. In case new nodes appear during processing, the Coordinator leaves the existing chunks alone and does not attempt to split them. The newly arrived node goes to the waiting list and can be assigned some of the undispatched chunks, or wait for a new processing batch.

Whenever a failure occurs in the Mapper node while the data processing takes place, the Coordinator recognizes such an occurrence through the heartbeat process that was introduced inearlier section The chunk that was supposed to be processed by the failed node is then re-scheduled right away and a new chunk_id number is appended to it, and finally passed over to another Mapper node.

4. Logic of the Mapper Node

The Mapper is the main workhorse of the entire distributed system. Each Mapper node gets assigned a portion of the input data by the Coordinator, parses each line of the logs to determine the severity level, counts their frequencies per severity, and maintains a list of unique messages per severity. Mappers have no information about the chunks processed by other Mappers – hence, the process of parsing is parallel and does not incur additional costs for coordinating.

4.1 Parsing algorithm

On reception of a chunk payload from the Sidecar, the Mapper parses the list of raw log lines and invokes parse_chunk function. The function loops through all the lines and searches for the severity keyword using a compiled regex expression. The operator is added to the regular pattern to ensure it matches whole words rather than substrings. To be safe, the re.IGNORECASE flag is set for case-insensitive matching.

The function then counts the number of lines parsed and saves the parsed lines to a set corresponding to the severity level found. Using a set in Python instead of a list for storing messages inherently eliminates duplicate log lines, which can be especially useful when dealing with system logs that often contain duplicate entries. After processing all lines in the chunk, the

function will return a dictionary object, which includes the final tally along with the distinct list of messages for each level.

```
import re

from collections import defaultdict


SEVERITY_PATTERN = re.compile(

    r'₩b(INFO|DEBUG|WARNING|ERROR|CRITICAL)₩b', re.IGNORECASE

)


def parse_chunk(lines: list[str]) -> dict:

    counts = defaultdict(int)

    messages = defaultdict(set)

    for line in lines:

        match = SEVERITY_PATTERN.search(line)

        if match:

            level = match.group(1).upper()

            counts[level] += 1

            messages[level].add(line.strip())

    return {

        'counts': dict(counts),

        'unique_messages': {k: list(v) for k, v in messages.items()}

    }
```

4.2 Internal Data Structure

After parsing, the Mapper holds its results in the following structure before dispatching to validators :

Severity	Count	Unique Messages (Sample)
INFO	1045	Service started, Connection established, ...
DEBUG	230	Cache miss on key X, Variable Y=null, ...
WARNING	75	Disk usage above 80%, Retry attempt 2, ...
ERROR	19	Connection refused, Timeout on host Z, ...
CRITICAL	3	Out of memory, Kernel panic, ...

The counts dictionary contains the essential numeric data that is pipelined through the validation and aggregation stage. The unique_messages dictionary acts as proof the Validators have the option to check sample unique messages and confirm that the Mapper's counts align with the actual contents of the log that the Mapper has been tasked with processing.

4.3 Concurrency Management

Every Mapper operates on its allotted chunk sequentially using a single thread. The reasoning behind this approach lies in the nature of log file parsing, which is a computation-intensive process; thus, adding concurrency within the Mapper itself will not provide any substantial gain but will complicate the synchronization process for no added benefit at the chunk level.

On the other hand, the Mapper's API implementation is concurrent by design and runs completely concurrently during the parse procedure. This means that the Mapper will be able to process requests such as chunk allocation, heartbeat response, and status check from the Coordinator without interrupting or queueing them due to the ongoing parse procedure.

In case the Coordinator notices that the Mapper has become silent while parsing data, the Mapper's asynchronous server will answer heartbeats regardless of whether the parse thread is busy at the time, avoiding misreporting of failures because of a busy parse thread looking unresponsive.

4.4 Sequence Diagram – Internal Mapper Node Processing

The following sequence diagram depicts the complete internal execution cycle of a Mapper node starting from the arrival of the chunk payload to the confirmation of the result after validation by the Validator quorum.

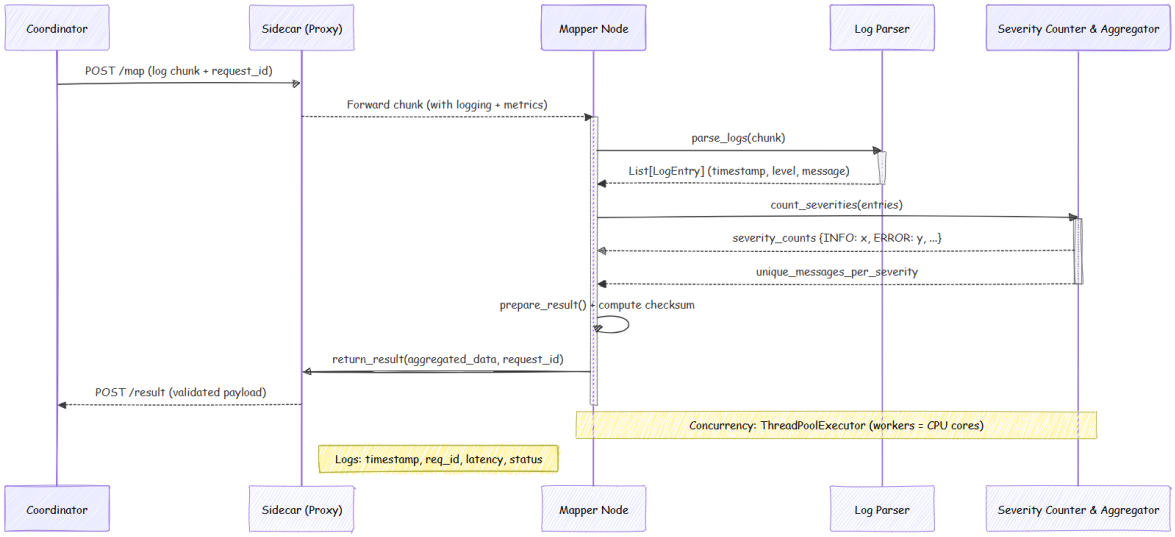

Diagram 1: Mapper Internal Execution (Sequence Diagram)

The process starts when the Coordinator sends a chunk to the Mapper's Sidecar using POST /map. The Sidecar logs the incoming request along with its timestamp and request ID, before passing the payload to the Mapper node's internal handler.

5. Validation Phase

This validation phase represents an additional verification step between the Mapper nodes and the Aggregator, which prevents any potentially incorrect or compromised data to go on to the final aggregation process. The Mapper sends its completed chunk result to at least two separate Validator nodes via the Sidecar at the same time, and each of these Validator nodes independently parses the raw log entries and compares their own calculated counts with the values submitted by the Mapper node.

The result is considered valid if the quorum, represented here by the formula floor(N/2) + 1, meaning 3 out of 4 Validator nodes in this particular case, returns an ACCEPT response back to

the Coordinator. Otherwise, the failed attempt gets repeated until the quorum is reached; should it fail a total of three times, the task gets re-assigned to a new Mapper node.

This consensus is performed by the Coordinator using an internal voting mechanism as opposed to the peer-based approach, making the procedure audit-safe and the Aggregator receiving only validated results stamped by the Coordinator. [6], [7]

6. Aggregation Phase

The aggregation phase is the last step of the distributed pipeline wherein the Aggregator node aggregates all the quorum-validated chunk results returned by the Validator nodes and generates a global severity count output. Aggregation is performed by the Aggregator progressively on incoming results through summation.

This process does not require the ordering of the arriving chunks, thanks to the commutative and associative properties of summation. To avoid receiving duplicate results from Sidecar due to retries or timeouts, each chunk is uniquely identified by the chunk_id field set by the Coordinator upon dispatch, and any result bearing the same chunk_id as one of the previous results is quietly dropped by the Aggregator.

While the system operates with eventual consistency while aggregating the chunks, strong consistency is enforced at the output generation phase where the final result is only delivered once the Coordinator has acknowledged the receipt and merging of all dispatched chunks through a finalize command, and hence a global output is generated, representing the severity counts of the entire collection of 1,372 log lines. [8], [9]

7. Sidecar Architecture

The Sidecar is a special-purpose component that accompanies each node in the cluster irrespective of its function and serves as the only entry point for all intra-cluster communications and offloads issues like logging, monitoring, retries, and circuit breaking from individual nodes' business logic based on the proven Sidecar Architecture pattern. All incoming and outgoing HTTP requests pass through the Sidecar, which logs each event in JSON format recording the timestamp, unique request ID, source and target nodes, endpoint address, response code, elapsed time, and retry counter, resulting in an exhaustive record of all the cluster's communications over time.

If outgoing requests result in errors, the Sidecar retries each connection using an exponential delay with up to three attempts, and once the failure count on the destination node reaches five consecutive times, the circuit breaker trips and blocks all future network calls to that peer for 30 seconds, after which it switches to the half-open probing mode.

It also abstracts all peer routing such that the logic of how the nodes work does not need to understand anything about the IP address, ports, and whether the other nodes are up and running or not because it only sends a call to a local internal API and the Sidecar takes care of everything else. [3]

7.1 Sidecar Log Entry Format

Each transaction that occurs within the Sidecar is captured as a one-liner JSON string and then written out to the local log file on the node, as well as stdout for log collection using Docker. Here's the schema for the log message format:

```
{
"timestamp": "2026-05-09T10:03:22.451Z",
 "request_id": "a3f7c2d1-...",
 "direction": "OUTGOING",
 "source_node": "mapper-1",
 "target_node": "validator-2",
 "endpoint": "POST /validate",
 "status_code": 200,
 "latency_ms": 47,
 "retry_count": 0,
 "circuit_state": "CLOSED",
 "payload_bytes": 1284,
 "outcome": "SUCCESS"
 }
```

All fields must be present; none can be NULL. The request_id is created by the source Sidecar and passed through unchanged at each hop, allowing for end-to-end tracing of an individual chunk from the Coordinator to the Aggregator.

7.2 Sidecar Failure Recovery Strategy

Sidecar's defense mechanism against the communication failures includes three layers of protection:

Layer 1: Exponential Backoff and Retry: In case of receiving a response code other than 2xx HTTP status codes or a connection timeout, the Sidecar will have to wait 200 milliseconds prior to attempting a first retry, 400 milliseconds before a second retry, and 800 milliseconds before a third retry attempt. Upon three attempts, if all the responses are non-2xx statuses, then it will be categorized as hard error, and the outcome parameter will have the value FAILURE.

Layer 2: Circuit Breaker: A per-peer failure counter is kept in memory. In case five successive calls to the same peer fail, the breaker will move from the CLOSED state to the OPEN state, and no more calls will be attempted to that peer until 30 seconds elapse from the time it was opened.

Layer 3 – Dead Letter Queue: In case the status of the circuit stays OPEN, and the node initiating the transmission is a Coordinator, the failed packet processing job is appended to an in-memory dead-letter queue. The coordinator's scheduler checks the queue every 10 seconds, attempting re-assignment to another healthy Mapper. [10],[11]

Circuit State	Behavior	Transition Condition
CLOSED	All calls forwarded normally	Default state
OPEN	Calls rejected immediately (fast fail)	5 consecutive failures
HALF_OPEN	One probe call allowed	30 s timeout after OPEN

7.3 How the Sidecar Improves Observability

Due to all inter-node traffic passing through the Sidecar, operators get one consistent source of truth for the whole cluster without having to individually monitor the business logic of each node. The list of advantages for observability includes:

End-to-end request tracing: By propagating the request_id, any log aggregation service (Grafana Loki, ELK stack, etc.) would be able to trace exactly what path was taken by a chunk from the Coordinator to the Aggregator node via retries, votes, and circuit breakers.Latency distribution: Due to recording the latency_ms in every entry, generating a histogram of latency (p50, p95, p99) by the endpoint helps find slow Validators or overwhelmed Mappers nodes.Circuit breaker status: Circuit breaker status makes it easy to know how connected the peers are. Operators would get notified immediately when a peer becomes open (indicative of issues before the coordinator marks the node as failed).

Retry Amplification Detection: Large retry_count values in the log stream denote problems in the network. Automated alarms for retry_count>1 assist in preventing retries from escalating into a

complete circuit breaker failure. Zero Code Instrumentation: The separation between the Sidecar and the other processes makes it easier to improve logging and metrics independently without making any modifications to the business logic of the Mapper, Validator, and Aggregator.

8. Sequence Diagram Communication Flow

Sequence diagrams give a clear graphical depiction of the order in which messages are sent between nodes within the distributed pipeline. Sequence diagrams presented in this section show four important communication paths, namely: cluster initialization, task assignment, the validator verification process, and finally the aggregator process. All these processes show how the system evolves from standalone processes to an organized cluster that delivers consistent results.

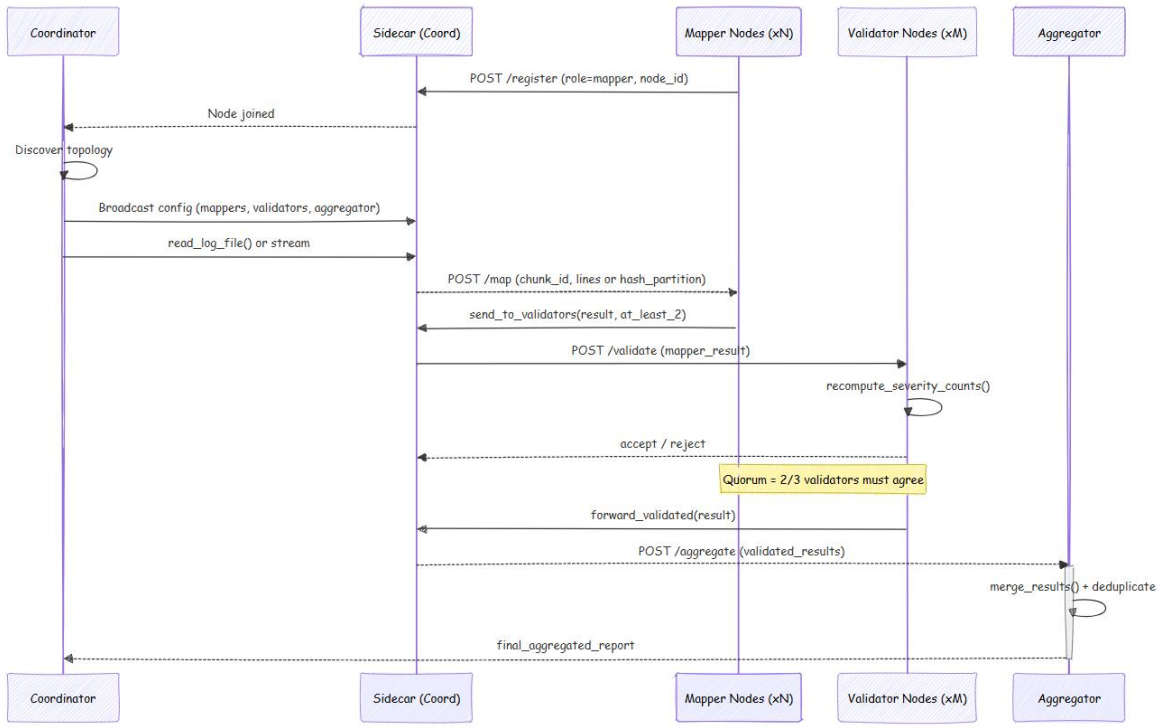

Diagram 2: Full System Communication Flow (Sequence Diagram)

five critical steps of the communication flow in a distributed system, highlighting how separate node processes work together to generate the final output. In the initialization step, each node that is not a coordinator registers with the Coordinator via POST /register through its Sidecar. When the required minimum number of roles are satisfied, the Coordinator announces the complete cluster topology and sets the cluster state to READY from INITIALISING. Then, the step of distributing work occurs; here, the Coordinator loads the input log file, partitions it into N

equal-sized partitions by round-robin batching, and POSTs each partition to the Mapper for processing via POST /map, keeping a record of all dispatching events in the completion ledger while checking for any failures in Mapper heartbeats. Once the Mapper parses the entire partition, the step of validation takes place; here, the Mapper Sidecar makes simultaneous POST /validate calls to at least two Validators, who independently re-parse the partition and provide ACCEPT/REJECT decisions on the partition to the Coordinator.

After obtaining the required number of ACCEPTs, the Coordinator sets the chunk to be valid and sends POST /aggregate to the Aggregator, providing the chunk_id as an idempotency key. The Aggregator updates its running totals of severity levels and responds with the latest partial values in the body of a 200 OK message. The finalization phase begins once the Coordinator's list of completed chunks, as recorded in its ledger, shows that all chunks have been processed. The Coordinator then sends POST /finalize to the Aggregator, who calculates and returns the final global severity count matrix, which the Coordinator publishes as the official output of the system. Once finalize is received, no further updates can be accepted by the Aggregator.

9. Failure Scenarios and Fault Tolerance Strategy

A distributed system should be built to ensure proper functioning even when a component itself fails. In this section, the major types of failures, their detection, and how the system recovers are described.

9.1 Failure of Mapper Node

Failure scenario: The Mapper node stops working while processing.

Detection: Lack of heartbeat is detected by the Coordinator; the Coordinator will flag the node as FAILED after 15 seconds (3 misses).

Recovery: The Coordinator will put the remaining chunk back into its queue with a different chunk_id, will remove the corresponding ledger entry, and will allocate it to another Mapper. Since the chunk_ids are unique, there will be no issues if the stale answer somehow reaches the Aggregator later on.

9.2 Failure of the Validator Node

Situation: The failure of the Validator node occurs during the validation process.

Identification: It involves the Sidecar's circuit breaker failing if there have been five failed attempts in a row to the Validator. Meanwhile, the Coordinator identifies a failed heartbeat in 15 seconds.

Resolution: The quorum formula, floor(N/2) + 1, is recalculated in relation to the number of live Validators left. If at least two Validators are still functioning, validation will proceed. However, validation cannot happen if quorum cannot be reached. In such cases, the chunk goes into a pending state. [12]

9.3 Aggregator Node Crash

Condition: The Aggregator crashes before finalization.

Detection: The Coordinator detects the missing heartbeat after 15 seconds.

Recovery: Since each submission of the result contains a chunk_id, the in-memory state of the Aggregator can be rebuilt using all the validated results in the completion ledger at the Coordinator. Once the Aggregator comes up, it will reregister itself, receive the topology, and the Coordinator will replay all the chunk results to the Aggregator.

9.4 Failure of Coordinator Node

Failure Scenario: The Coordinator node fails.

Effect of this failure: This failure scenario is one of the most important failures because it affects the whole topology and task assigning in the current architecture. Mitigation: Currently, the Coordinator updates its topology registry and completion registry to a persistent JSON file every five seconds. When the Coordinator stops functioning, the watchdog service (which runs the Coordinator with the restart policy of Docker compose set to 'always') starts up the Coordinator again. It loads the persisted information and continues processing any incomplete chunk assignments. Future work will move toward a leader-based architecture with multiple Coordinators that implement the Raft consensus protocol. [16]

9.5 Network Partition

Scenario: Nodes within a segment become temporarily disconnected owing to network partitioning.

Detection: Both the circuit breaker of Sidecar and the heartbeat mechanism of Coordinator independently detect nodes that have become unreachable.

Recovery: If nodes were within the partition, they would continue to respond to heartbeats locally until the partition resolves and they reconnect or begin sending heartbeats again. Any

chunks allocated to nodes within the isolated partition would be added back to the queue after crossing the FAILED threshold.

10. Scalability Strategy

The design enables horizontal scalability: performance scales linearly with the number of Mapper and Validator nodes. The following paragraphs detail the strategies enabling this behavior.

10.1 Horizontal Scaling of Mapper Nodes

Launching a new Mapper node involves creating a container with ROLE=mapper and a valid COORDINATOR_URL. The node will auto-register itself, retrieve the current topology, and become immediately eligible for chunk allocation. The Coordinator will dynamically calculate new chunk sizes using the number of active Mapper nodes for future rounds. As Mapper nodes are stateless within each batch (each chunk is independent), no state transfer is needed.

10.2 Horizontal Scaling of Validator Nodes

Validators are also stateless, receiving a chunk from the raw log, recomputing severity counts, and voting to accept/reject. Adding more validators to the network improves quorum reliability without reconfiguring the entire topology since the Coordinator calculates floor(N/2)+1 dynamically per validation round. [8]

10.3 Coordinator Throughput

The Coordinator is the only centrally placed component, thus becoming the first bottleneck when it comes to scaling. Its throughput depends on how fast it can dispatch chunks and aggregate their results through HTTP requests. Currently, for the given size of the clusters (few tens of nodes, log files containing millions of entries) this isn't an issue. However, in the case of larger clusters, the following optimizations could be applied: (a) batched chunk dispatch – instead of making a single HTTP request per each chunk, pass multiple chunks' descriptions within a single request body; (b) pipelined dispatch – parallelize chunk dispatching with result aggregation so that the Coordinator isn't forced to wait for results from one mapper to dispatch another.

10.4 Aggregator Throughput

The Aggregator gets one POST /aggregate request per each validated chunk. As the summing function has O(S) complexity where S is the number of severity levels (set to 5) it runs in constant time independently from the cluster size. Thus, it doesn't become a bottleneck for any number of Mappers.

10.5 Scalability Challenges and Open Research Areas

Our present architecture ensures practical scalability of Mapper and Validator nodes. These challenges have been determined:

Single Coordinator: Having a centralized topology manager results in limited scalability beyond approximately 50-100 nodes due to increased computational costs associated with heartbeats processing. Solution: decentralizing coordinator with Raft leader election.

Single Aggregator: There can be only one Aggregator within each cluster. When dealing with extremely high log sizes, building an aggregate hierarchy (with local Aggregators submitting to the global one) would help mitigate the problem.

In-memory state: Ledger storage in the Coordinator and running aggregations in the Aggregator are performed in memory. Introducing Redis-based shared storage will ensure state persistence and scalable Coordinator infrastructure.

Reference

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," Communications of the ACM, vol. 51, no. 1, pp. 107–113, Jan. 2008.

[2] G. Agha, Actors: A Model of Concurrent Computation in Distributed Systems. Cambridge, MA: MIT Press, 1986.

[3] B. Burns and D. Oppenheimer, "Design patterns for container-based distributed systems," in Proc. 8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud), Denver, CO, Jun. 2016.

[4] A. S. Tanenbaum and M. Van Steen, Distributed Systems: Principles and Paradigms, 3rd ed. Pearson, 2017.

[5] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," Journal of the ACM, vol. 43, no. 2, pp. 225–267, Mar. 1996.

[6] D. K. Gifford, "Weighted voting for replicated data," in Proc. 7th ACM Symposium on Operating Systems Principles (SOSP), Pacific Grove, CA, Dec. 1979, pp. 150–162.

[7] L. Lamport, "The part-time parliament," ACM Transactions on Computer Systems, vol. 16, no. 2, pp. 133–169, May 1998.

[8] W. Vogels, "Eventually consistent," Communications of the ACM, vol. 52, no. 1, pp. 40–44, Jan. 2009.

[9] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," ACM SIGACT News, vol. 33, no. 2, pp. 51–59, Jun. 2002.

[10] M. T. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018.

[11] M. Fowler, "CircuitBreaker," martinfowler.com, Mar. 2014. [Online]. Available: https://martinfowler.com/bliki/CircuitBreaker.html

[12] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in Proc. USENIX Annual Technical Conference (ATC), Philadelphia, PA, Jun. 2014, pp. 305–319.

[13] S. He, J. Zhu, P. He, and M. R. Lyu, "Loghub: A large collection of system log datasets towards automated log analytics," arXiv preprint arXiv:2008.06448, Aug. 2020.

[14] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," Linux Journal, vol. 2014, no. 239, Mar. 2014.

[15] P. A. Bernstein, V. Hadzilacos, and N. Goodman, Concurrency Control and Recovery in Database Systems. Reading, MA: Addison-Wesley, 1987.

[16] S. Ramírez, "FastAPI Documentation," 2023. [Online]. Available: https://fastapi.tiangolo.com/

[17] Python Software Foundation, "asyncio — Asynchronous I/O," Python 3.11 Documentation, 2023. [Online]. Available: https://docs.python.org/3/library/asyncio.html